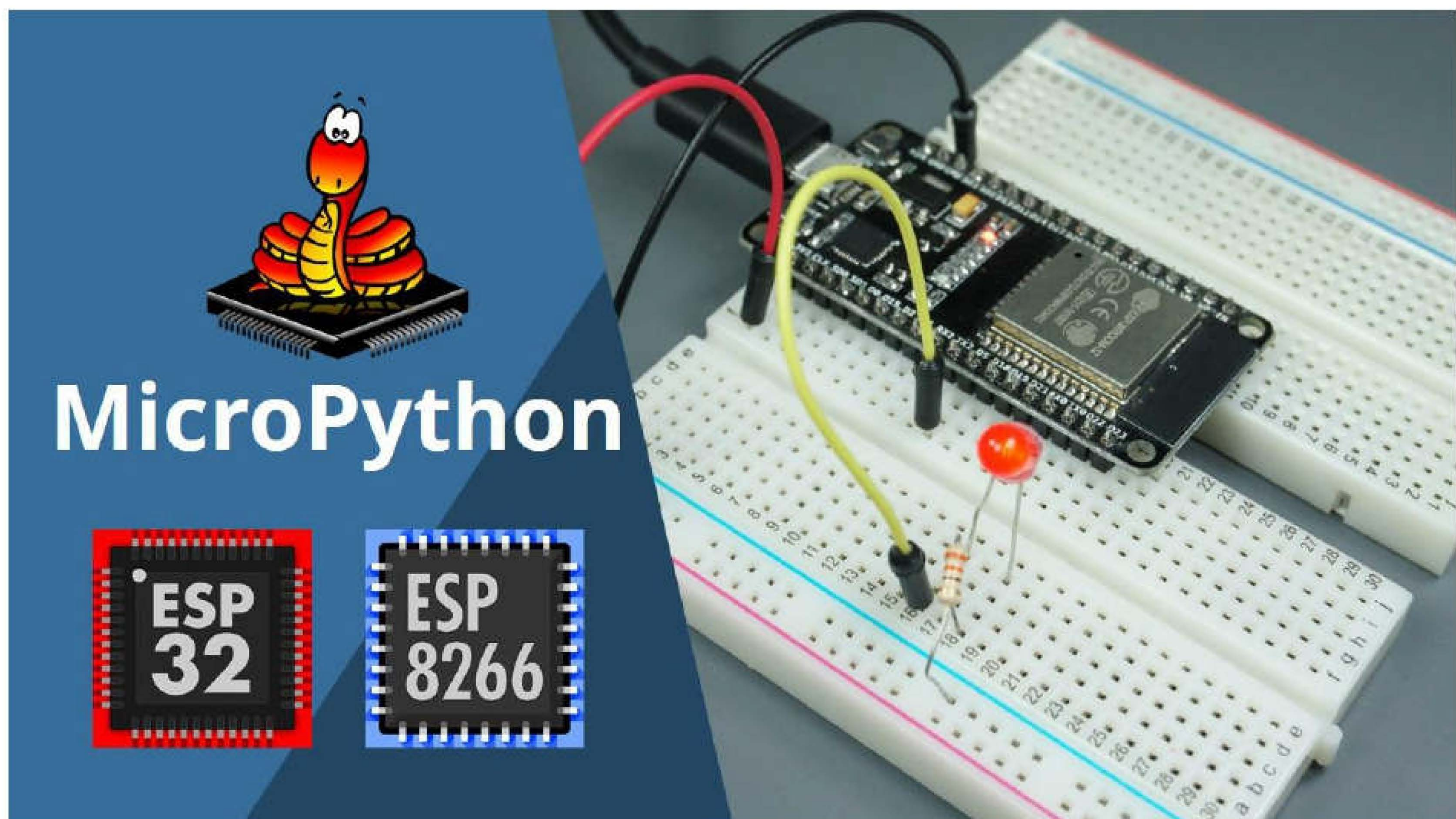


PWM – Pulse Width Modulation

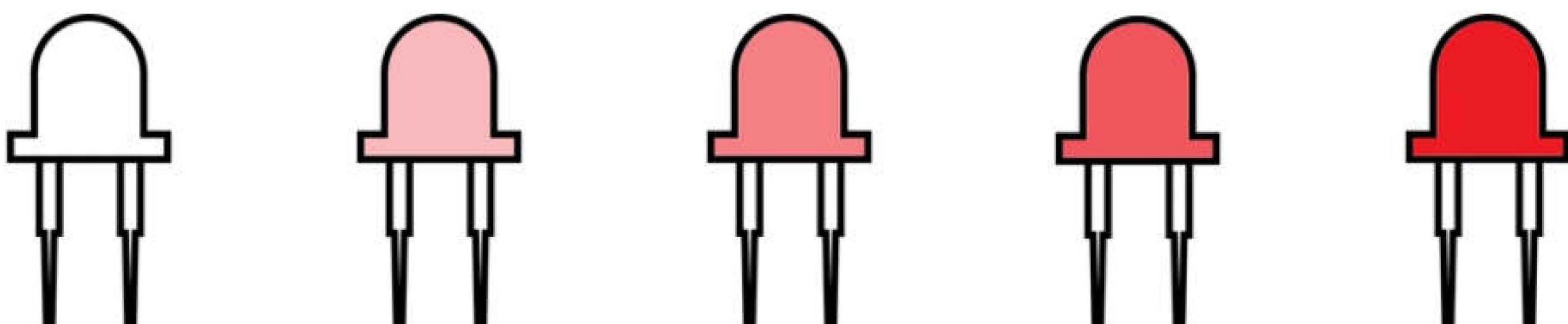


In this Unit, you'll learn how to dim an LED using PWM. We'll also build a simple project in which you can control the LED brightness using a potentiometer.

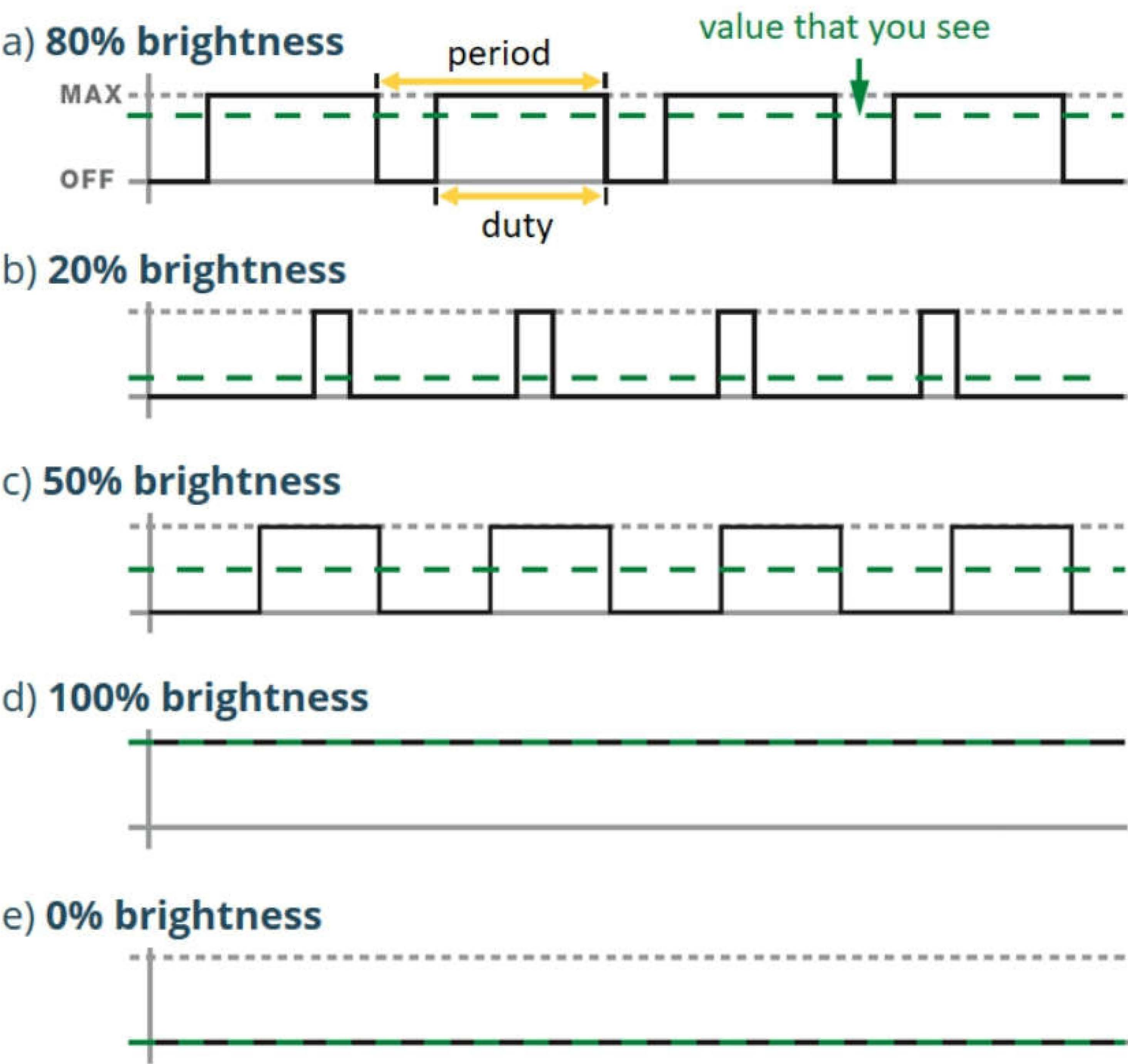
Pulse-Width Modulation

The ESP32/ESP8266 GPIOs can be set either to output 0V or 3.3V, but they can't output any voltages in between (apart from the DAC pins on the ESP32). However, you can output "fake" mid-level voltages using pulse-width modulation (PWM), which is how you'll produce varying levels of LED brightness for this project.

If you alternate an LED's voltage between HIGH and LOW very fast, your eyes can't keep up with the speed at which the LED switches on and off; you'll simply see some gradations in brightness.



That’s basically how PWM works—by producing an output that changes between HIGH and LOW at a very high frequency. The duty cycle is the fraction of the time period at which the LED is set to HIGH. The following figure illustrates how PWM works.



For instance, a duty cycle of 50 percent results in 50 percent LED brightness, a duty cycle of 0 means the LED is fully off, and a duty cycle of 100 means the LED is fully on. Changing the duty cycle is how you produce different levels of brightness, and that’s what we’re going to do here.

Schematic

For this example, wire an LED to your ESP board. We'll wire the LED to GPIO 5 in both boards, but you can choose another suitable PWM pin. All ESP pins that can act as outputs can generate PWM signals.

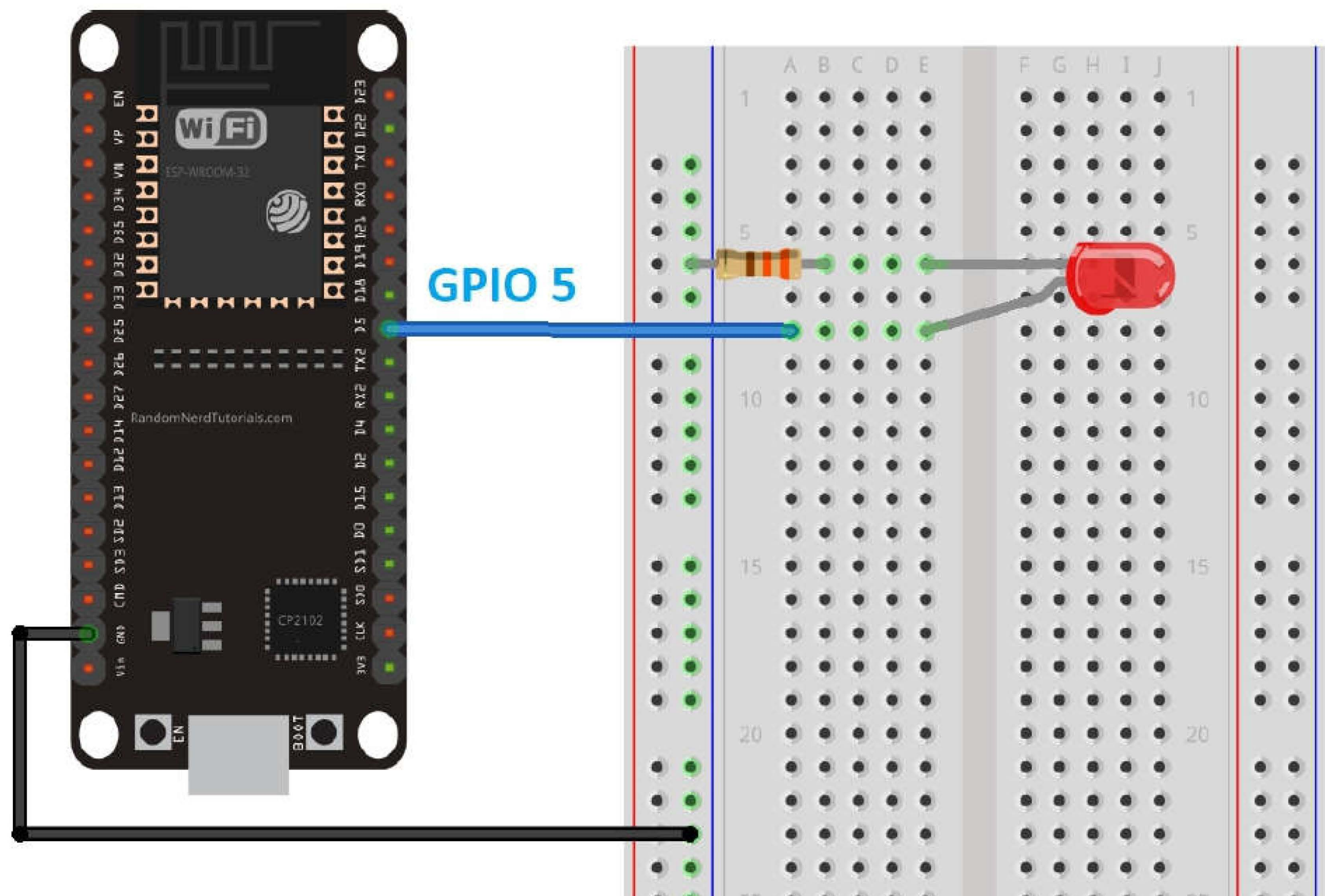
Parts required

Here's a list of parts you need to build the circuit:

- [ESP32](#) or [ESP8266](#)
- [5mm LED](#)
- [330 Ohm resistor](#)
- [Breadboard](#)
- [Jumper wires](#)

Schematic – ESP32

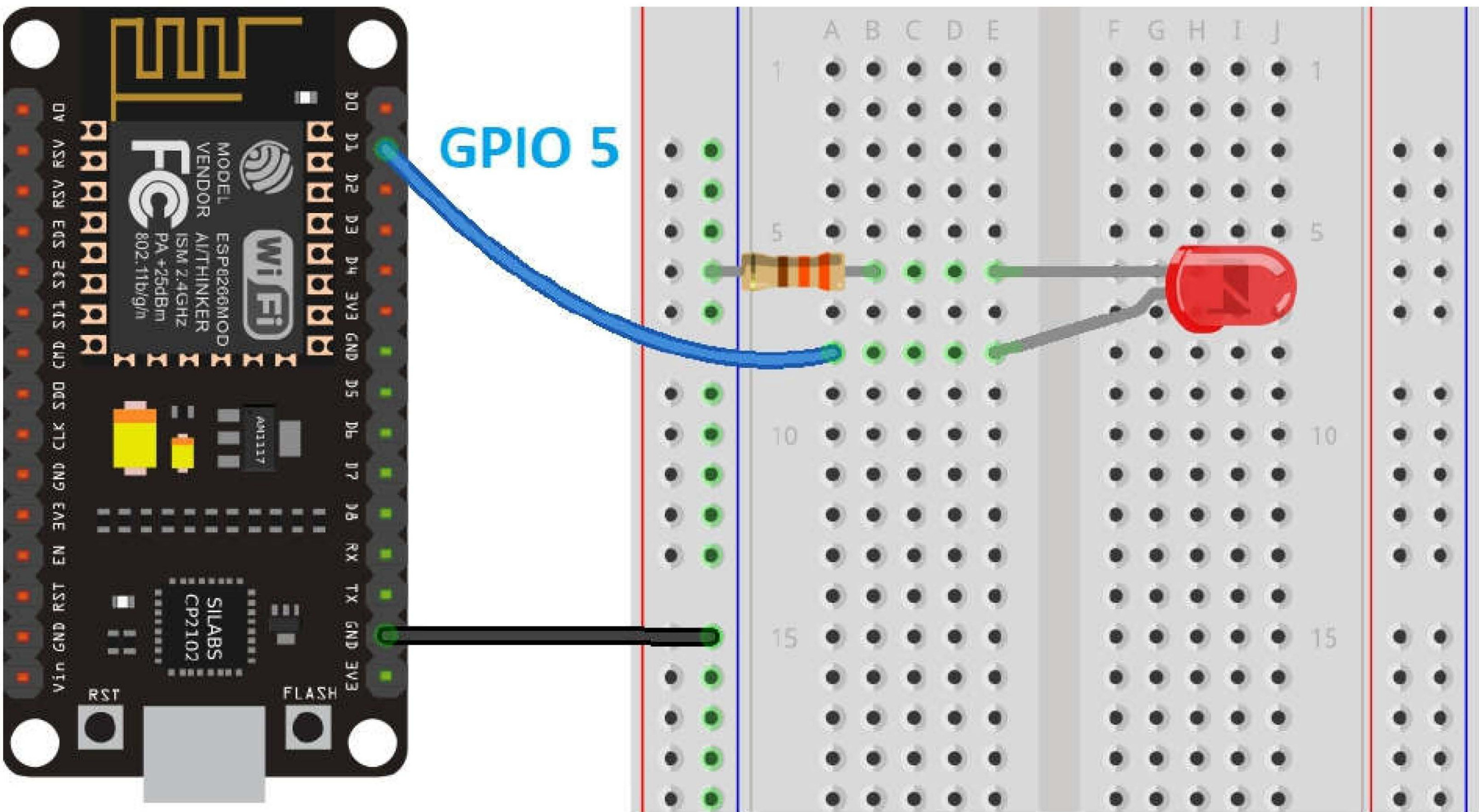
Follow the next schematic diagram if you're using an ESP32 board:



(This schematic uses the ESP32 DEVKIT DOIT board version with 36 GPIOs. Before assembling the circuit double-check the pinout for the board you're using.)

Schematic – ESP8266

Follow the next schematic diagram if you’re using an ESP8266 board:



Script

Here’s the script that changes the LED brightness over time by increasing the duty cycle. This script works with the ESP32 and ESP8266.

```
from machine import Pin, PWM
from time import sleep

frequency = 5000
led = PWM(Pin(5), frequency)

while True:
    for duty_cycle in range(0, 1024):
        led.duty(duty_cycle)
        sleep(0.005)
```

SOURCE CODE

https://github.com/RuiSantosdotme/ESP-MicroPython/blob/master/code/GPIOs/PWM/pwm_basic.py

How the code works

To create a PWM pin, import the `PWM` class in addition to the `Pin` class from the `machine` module.

```
from machine import Pin, PWM
```

Then, create a `PWM` object called `led`.

```
led = PWM(Pin(5), frequency)
```

To create a `PWM` object, you need to pass as parameters, the pin it is connected to, the signal frequency and the duty cycle.

- **Frequency:** The frequency can be a value between 0 and 78125. A frequency of 5000 Hz can be used to control an LED brightness.
- **Duty cycle:** The duty cycle can be a value between 0 and 1023. In which 1023 corresponds to 100% duty cycle (full brightness), and 0 corresponds to 0% duty cycle (unlit LED).

We'll just set the duty cycle on the `while` loop, so we don't need to pass the duty cycle parameter. If you don't set the duty cycle when instantiating the `PWM` object, it will be 0 by default.

To set the duty cycle use the `duty()` method on the `PWM` object and pass the duty cycle as an argument:

```
led.duty(duty_cycle)
```

Inside the `while` loop, we create a `for` loop that increases the duty cycle by 1 in each loop with an interval of 5 ms between each change.

The `range()` function has the following syntax:

```
range(start, stop)
```

- **Start:** a number that specifies at which position to start. We want to start with 0 duty cycle;
- **Stop:** a number that specifies at which position we want to stop, excluding that value. The maximum duty cycle is 1023, because we are incrementing 1 in each loop, the last value should be 1023+1. So, we'll use 1024.

In each `for` loop, we print the current value of the duty cycle and set the LED's duty cycle to the current `duty_cycle` value:

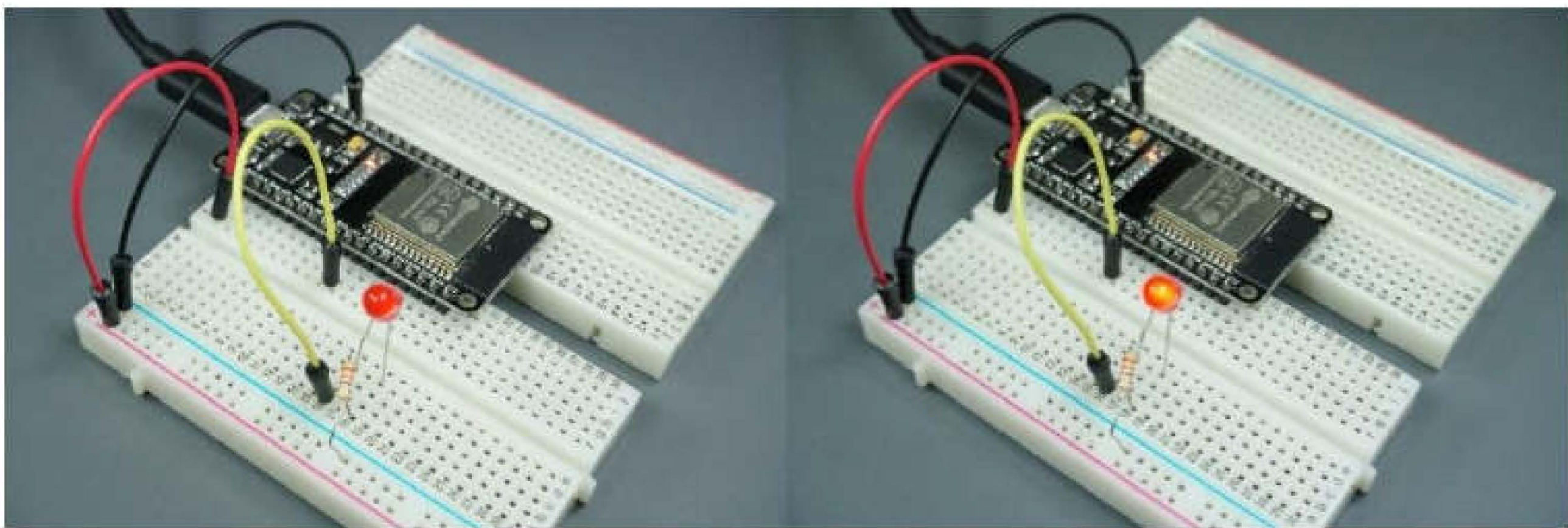
```
led.duty(duty_cycle)
```

After that, the `duty_cycle` is incremented by 1. The following table helps you understand what's happening in the code.

	duty_cycle	led.duty(duty_cycle)	GPIO state
loop 1	0	led.duty(0)	0% brightness
loop 2	1	led.value(1)	~ 0.1% brightness
loop 3	2	led.value(2)	~ 0.2% brightness
...	...	...	...
loop 1024	1023	led.value(1023)	100% brightness

Demonstration

Upload the code to your ESP32 or ESP8266. The LED attached to GPIO 5 should increase brightness over time.



LED Dimmer Switch Project

We can apply the concepts learned previously to build an LED dimmer switch: control the LED brightness using a potentiometer.

Schematic

For this project you need to wire a potentiometer to GPIO 34 (ESP32) or ADC 0 (ESP8266) and an LED to GPIO 5.

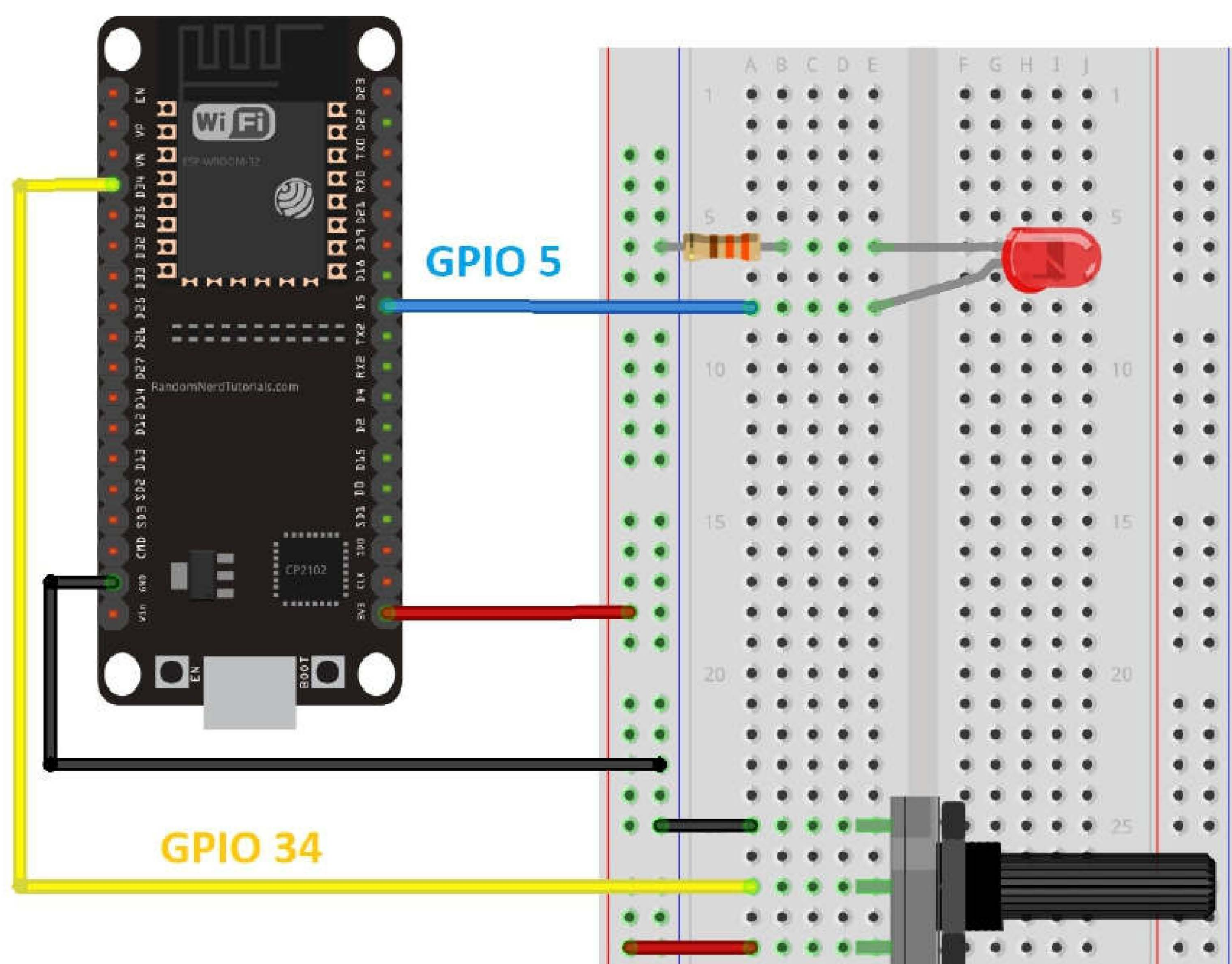
Parts required

Here's a list of parts you need to build the circuit:

- [ESP32](#) or [ESP8266](#)
- [5mm LED](#)
- [330 Ohm resistor](#)
- [Potentiometer](#)
- [Breadboard](#)
- [Jumper wires](#)

Schematic – ESP32

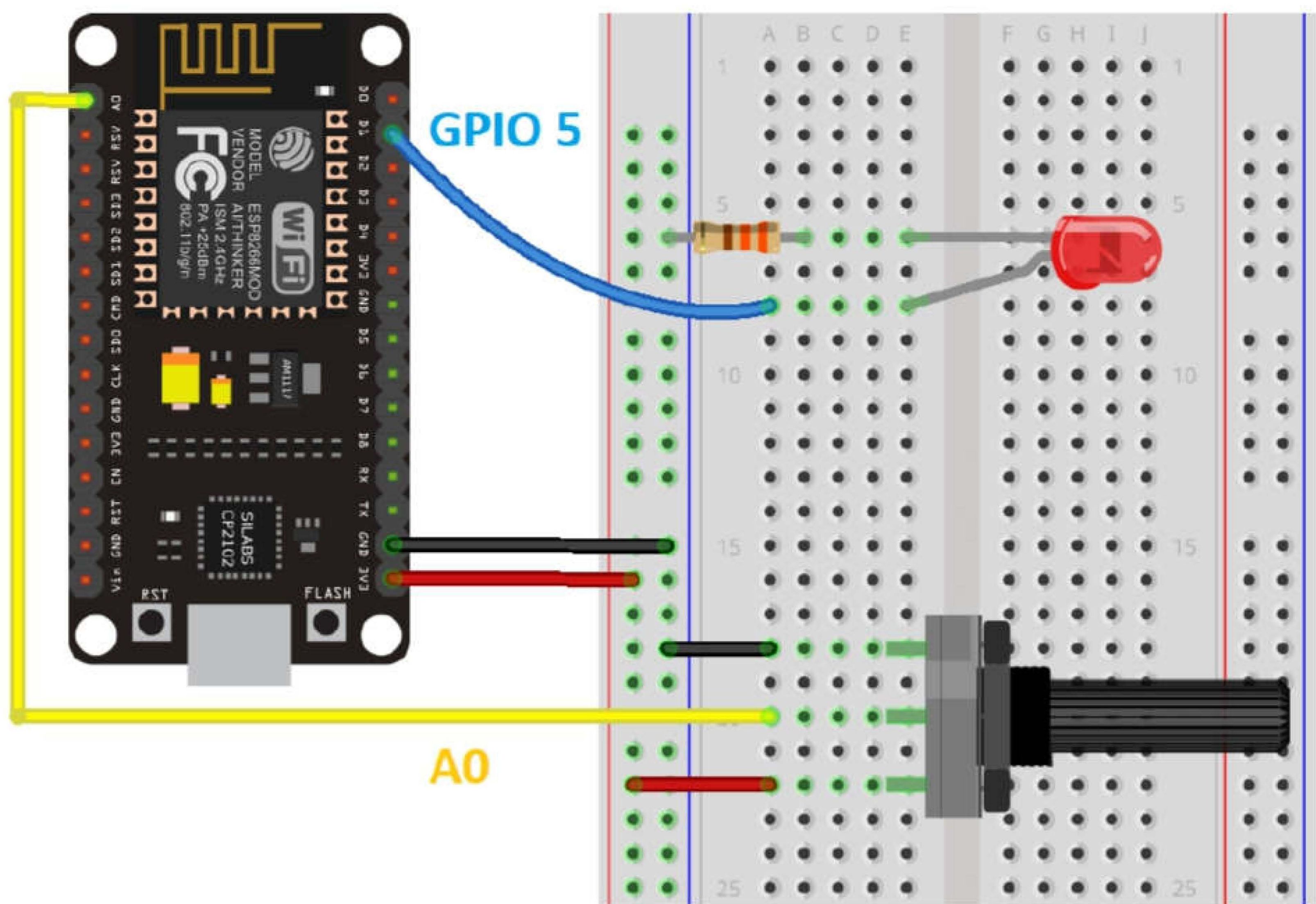
Follow the next schematic diagram if you're using an ESP32 board:



(This schematic uses the ESP32 DEVKIT DOIT board version with 36 GPIOs. Before assembling the circuit double-check the pinout for the board you're using.)

Schematic – ESP82

Follow the next schematic diagram if you’re using an ESP8266 board:



Script

Because ADC works differently in ESP32 and ESP8266, we have a different script for each board.

Script - ESP32

Copy the following code to the *main.py* if you’re using an ESP32 board.

```
from machine import Pin, PWM, ADC
from time import sleep

frequency = 5000
led = PWM(Pin(5), frequency)
pot = ADC(Pin(34))
pot.width(ADC.WIDTH_10BIT)
pot.atten(ADC.ATTN_11DB)

while True:
    pot_value = pot.read()
    print(pot_value)
```



```
if pot_value < 15:
    led.duty(0)
else:
    led.duty(pot_value)

sleep(0.1)
```

SOURCE CODE

https://github.com/RuiSantosdotme/ESP-MicroPython/blob/master/code/GPIOs/PWM_Pot/pwm_pot_esp32.py

How the code works

As we've seen previously, we need to import the `Pin`, `PWM`, and `ADC` class from the `machine` module, and the `sleep` class from the `time` module.

```
from machine import Pin, PWM, ADC
from time import sleep
```

Define the frequency of the PWM signal and create a `PWM` object called `led` on GPIO 5.

```
frequency = 5000
led = PWM(Pin(5), frequency)
```

Define an `ADC` object on pin 34, the GPIO the potentiometer is connected to.

```
pot = ADC(Pin(34))
```

In this specific example, we'll set the analog reading to 10-bit resolution. This simplifies our code, because this way we have the same range of values (0-1023) for both the analog reading, and the duty cycle.

```
pot.width(ADC.WIDTH_10BIT)
```

We want to be able to read full range (0-3.3V), so we need to add the following line:

```
pot atten(ADC.ATTN_11DB)
```

In the `while` loop, we read the `pot` value, save it in the `pot_value` variable and print the value on the Shell:

```
pot_value = pot.read()
print(pot_value)
```


Then we have a condition that ensures the LED turns off with small potentiometer readings. If the `pot` value is smaller than 15, we set the duty cycle to 0.

```
if pot_value < 15:
    led.duty(0)
```

As seen previously, to set the duty cycle we simply need to use the `duty()` method on a PWM object, in this case the `led` variable.

```
led.duty(pot_value)
```

If the potentiometer reading is more than 15, we'll set the reading from the potentiometer as the duty cycle:

```
else:
    led.duty(pot_value)
```

Script – ESP8266

Copy the following code to the `main.py` if you're using an ESP8266 board.

```
from machine import Pin, PWM, ADC
from time import sleep

frequency = 5000
led = PWM(Pin(5), frequency)
pot = ADC(0)

while True:
    pot_value = pot.read()
    print(pot_value)

    if pot_value < 15:
        led.duty(0)
    else:
        led.duty(pot_value)

    sleep(0.1)
```

SOURCE CODE

https://github.com/RuiSantosdotme/ESP-MicroPython/blob/master/code/GPIOs/PWM_Pot/pwm_pot_esp8266.py

How the code works

As we've seen previously, we import the `Pin`, `PWM`, and `ADC` class from the `machine` module, and the `sleep` class from the `time` module.

```
from machine import Pin, PWM, ADC
from time import sleep
```

Define the frequency of the PWM signal, and create a `PWM` object called `led` on GPIO 5:

```
frequency = 5000
led = PWM(Pin(5), frequency)
```

Define an `ADC` object on the ESP8266 analog pin (A0).

```
pot = ADC(0)
```

In the `while` loop, we read the `pot` value, save it in the `pot_value` variable and print the value on the Shell:

```
pot_value = pot.read()
print(pot_value)
```

We have a condition that ensures the LED turns off with small potentiometer readings. If the `pot` value is smaller than 15, we set the duty cycle to 0.

```
if pot_value < 15:
    led.duty(0)
```

As explained previously, to set the duty cycle, use the `duty()` method on a `PWM` object, in this case the `led` variable.

```
led.duty(pot_value)
```

If the potentiometer reading isn't smaller than 15, we'll set the reading from the potentiometer as the duty cycle:

```
else:
    led.duty(pot_value)
```


Demonstration

Upload the code to your ESP32 or ESP8266 board. Then, you should be able to control the LED brightness by rotating the potentiometer. The readings from the potentiometer are printed on the Shell.

